

Exploratory Data Analysis (EDA)

EDA Process Overview

1. Definition & Importance

Exploratory Data Analysis (EDA) is the critical process of performing initial investigations on data to discover patterns, spot anomalies, test hypotheses, and check assumptions with the help of summary statistics and graphical representations.

Why EDA is Crucial:

- Understand data structure and quality
- Detect errors and outliers
- Identify important variables and relationships
- Formulate hypotheses for further testing
- Guide feature engineering decisions
- Validate data for machine learning models

2. Comprehensive EDA Process

Phase 1: Data Collection & Initial Inspection

python

Key Operations:

```
df.shape      # Dataset dimensions
df.info()     # Data types and memory usage
df.head()     # First few rows
df.tail()     # Last few rows
df.columns    # Column names
df.dtypes     # Data types per column
```

What to Look For:

- Total number of observations and features
- Data types appropriateness (e.g., numeric vs categorical)
- Initial data quality assessment
- Understanding the business context of each variable

Phase 2: Data Quality Assessment

Missing Values Analysis:

- Identify columns with missing data
- Calculate percentage of missing values
- Understand pattern of missingness (MCAR, MAR, MNAR)
- Document missing data impact

Duplicate Data:

- Check for duplicate rows
- Identify partial duplicates
- Decide on deduplication strategy

Data Integrity Checks:

- Validate value ranges (e.g., age shouldn't be negative)
- Check for inconsistent formatting
- Verify categorical value consistency

Phase 3: Univariate Analysis

- Analyze each variable independently
- Understand distributions and spreads
- Identify outliers and unusual values

Phase 4: Bivariate/Multivariate Analysis

- Explore relationships between variables
- Identify correlations and interactions
- Discover patterns across multiple dimensions

Phase 5: Documentation & Reporting

- Summarize key findings
- Document data quality issues
- Provide recommendations for data cleaning
- Suggest features for modeling

3. EDA Tools & Techniques

Statistical Summaries:

- Descriptive statistics (mean, median, mode, std, etc.)
- Quantiles and percentiles
- Value counts for categorical data

Visualization Methods:

- Distribution plots (histograms, box plots)
 - Relationship plots (scatter plots, heatmaps)
 - Composition charts (pie charts, stacked bars)
 - Comparison charts (bar charts, line plots)
-

Handling Categorical vs Numerical Data

1. Categorical Data - Deep Dive

Types of Categorical Data:

A. Nominal Data:

- No intrinsic ordering or ranking
- Examples: Gender, Country, Product Category, Color
- **Analysis Focus:** Frequency distribution, proportions

B. Ordinal Data:

- Natural ordered categories
- Examples: Education Level, Customer Rating, Income Bracket
- **Analysis Focus:** Order-based analysis, median, percentiles

C. Binary Data:

- Special case with only two categories
- Examples: Yes/No, True/False, 0/1
- **Analysis Focus:** Proportion analysis, logistic relationships

Categorical Data Analysis Techniques:

Descriptive Statistics:

- **Frequency Counts:** `df['column'].value_counts()`
- **Proportions:** `df['column'].value_counts(normalize=True)`
- **Mode:** Most frequent category
- **Cardinality:** Number of unique categories

Visualization Methods:

- **Bar Charts:** For frequency distribution
- **Pie Charts:** For proportion representation (use sparingly)
- **Tree Maps:** For hierarchical categorical data
- **Stacked Bar Charts:** For comparing categories across groups

Common Challenges with Categorical Data:

High Cardinality:

- **Problem:** Too many unique categories (e.g., zip codes, user IDs)
- **Solutions:**
 - Group rare categories into "Other"
 - Use target encoding for modeling
 - Apply clustering on categories

Inconsistent Formatting:

- **Problem:** Same category represented differently ("Male" vs "M" vs "male")

- Solutions:** Standardization, string cleaning

Missing Categories:

- Problem:** Important categories missing from training data
- Solutions:** Ensure representative sampling

2. Numerical Data - Comprehensive Analysis

Types of Numerical Data:

A. Discrete Data:

- Countable numerical values
- Examples: Number of children, Items purchased, Page views
- Characteristics:** Finite number of values, gaps between values

B. Continuous Data:

- Measurable numerical values
- Examples: Height, Weight, Temperature, Time
- Characteristics:** Infinite possible values, no gaps

Numerical Data Analysis Techniques:

Measures of Central Tendency:

- Mean:** Average value (sensitive to outliers)
- Median:** Middle value (robust to outliers)
- Mode:** Most frequent value (for discrete data)

Measures of Spread/Dispersion:

- Range:** Difference between max and min
- Variance:** Average squared deviation from mean
- Standard Deviation:** Square root of variance
- IQR (Interquartile Range):** Range between 25th and 75th percentiles
- MAD (Mean Absolute Deviation):** Average absolute deviation

Distribution Shape Analysis:

- Skewness:** Measure of asymmetry
 - Positive skew: Right-tailed distribution
 - Negative skew: Left-tailed distribution
- Kurtosis:** Measure of tail heaviness
 - High kurtosis: Heavy tails, more outliers
 - Low kurtosis: Light tails, fewer outliers

Key Percentiles & Quantiles:

- Quartiles:** Q1 (25%), Q2 (50% - median), Q3 (75%)
- Deciles:** Every 10th percentile
- Percentiles:** Specific percentage points

Advanced Numerical Analysis:

Outlier Detection Methods:

- Z-score method:** Values beyond ± 3 standard deviations
- IQR method:** Values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$
- Visual methods:** Box plots, scatter plots

Normality Testing:

- Visual:** Q-Q plots, histograms with normal curve
- Statistical tests:** Shapiro-Wilk, Kolmogorov-Smirnov
- Impact:** Many statistical methods assume normality

Visualization Techniques for Numerical Data:

Distribution Plots:

- Histograms:** For understanding frequency distribution
- Density Plots:** Smooth version of histograms
- Box Plots:** For median, quartiles, and outliers
- Violin Plots:** Combination of box plot and density plot

Summary Visualization:

- Summary Statistics Table:** Comprehensive numerical summary

•**Five-number Summary:** Min, Q1, Median, Q3, Max

3. Practical Analysis Framework

For Each Variable, Ask:

- 1.**What type of data is this?** (Categorical/Numerical, Nominal/Ordinal, Discrete/Continuous)
- 2.**What is the distribution?** (Normal, Skewed, Bimodal, Uniform)
- 3.**Are there missing values?** (How many? What pattern?)
- 4.**Are there outliers?** (How many? How extreme?)
- 5.**What are the key statistics?** (Central tendency, spread, frequencies)
- 6.**What is the business meaning?** (Context and importance)

Data Quality Metrics to Track:

- Completeness:** Percentage of non-missing values
 - Uniqueness:** Number of duplicate records
 - Consistency:** Adherence to expected formats and ranges
 - Accuracy:** How well data represents real-world values
 - Validity:** Conformity to business rules and constraints
-

Advanced Topics

Feature Engineering (Encoding, Scaling)

1. Understanding Feature Engineering

Definition: Feature engineering is the process of transforming raw data into features that better represent the underlying problem to predictive models, resulting in improved model accuracy on unseen data.

Why Feature Engineering Matters:

- Machine learning algorithms require numerical input
- Proper feature representation can significantly improve model performance
- Helps algorithms learn patterns more effectively
- Reduces computational complexity
- Handles missing values and outliers appropriately

2. Handling Categorical Data: Encoding Techniques

A. Nominal Encoding Methods

1. One-Hot Encoding:

```
python  
  
# Creates binary columns for each category  
pd.get_dummies(df, columns=['category_column'])
```

- Pros:** No artificial ordinal relationship, works well with most algorithms
- Cons:** Can create high dimensionality (curse of dimensionality)
- Use Case:** Low cardinality nominal data, tree-based models

2. Label Encoding:

```
from sklearn.preprocessing import LabelEncoder  
le = LabelEncoder()  
df['encoded_column'] = le.fit_transform(df['category_column'])
```

- Pros:** Simple, maintains single column
- Cons:** Creates artificial ordinal relationship
- Use Case:** Ordinal data, tree-based models that can handle ordinal relationships

3. Binary Encoding:

- Converts categories to binary code, then splits into separate columns
- Pros:** Reduces dimensionality compared to one-hot
- Cons:** Less interpretable
- Use Case:** High cardinality categorical data

4. Frequency Encoding:

```
frequency_map = df['category_column'].value_counts().to_dict()
df['frequency_encoded'] = df['category_column'].map(frequency_map)
```

- Pros:** No dimensionality increase, captures category prevalence
- Cons:** Loses category identity, same frequency categories get same encoding
- Use Case:** High cardinality, when frequency is informative

5. Target Encoding (Mean Encoding):

```
python

# Encode categories with mean of target variable
target_mean = df.groupby('category_column')['target'].mean()
df['target_encoded'] = df['category_column'].map(target_mean)
```

- Pros:** Powerful, captures relationship with target
- Cons:** Risk of overfitting, requires careful validation
- Use Case:** High cardinality, strong relationship expected with target

B. Ordinal Encoding

Manual Ordinal Encoding:

```
# Define custom mapping for ordered categories
education_map = {'High School': 1, 'Bachelor': 2, 'Master': 3, 'PhD': 4}
df['education_encoded'] = df['education'].map(education_map)
```

3. Handling Numerical Data: Scaling & Transformation

A. Scaling Techniques

1. Standardization (Z-score Normalization):

```
python

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df['scaled_column'] = scaler.fit_transform(df[['numerical_column']])
```

- Formula:** $(x - \text{mean}) / \text{std}$
- Result:** Mean = 0, Standard Deviation = 1
- Use Case:** Algorithms assuming normal distribution (SVM, Logistic Regression, PCA)

2. Min-Max Scaling (Normalization):

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df['normalized_column'] = scaler.fit_transform(df[['numerical_column']])
```

- Formula:** $(x - \text{min}) / (\text{max} - \text{min})$
- Result:** Range [0, 1]
- Use Case:** Neural networks, algorithms sensitive to feature scales

3. Robust Scaling:

```
python

from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
df['robust_scaled'] = scaler.fit_transform(df[['numerical_column']])
```

- Uses:** Median and IQR instead of mean and std
- Use Case:** Data with outliers, when outliers shouldn't influence scaling

4. MaxAbs Scaling:

```
python

from sklearn.preprocessing import MaxAbsScaler
scaler = MaxAbsScaler()
df['maxabs_scaled'] = scaler.fit_transform(df[['numerical_column']])
```

- Scales each feature by its maximum absolute value**
- Result:** Range [-1, 1]
- Use Case:** Sparse data, data centered around zero

B. Numerical Transformations

1. Logarithmic Transformation:

```
python

df['log_transformed'] = np.log1p(df['numerical_column']) # log(1+x)
```

- Use Case:** Right-skewed data, multiplicative relationships

2. Square Root Transformation:

python

```
df['sqrt_transformed'] = np.sqrt(df['numerical_column'])
```

- Use Case:** Moderate right skew, count data

3. Box-Cox Transformation:

python

```
from scipy import stats
```

```
df['boxcox_transformed'], _ = stats.boxcox(df['numerical_column'] + 1)
```

- Use Case:** Making data more normal distribution-like

4. Binning/Discretization:

python

```
# Equal-width binning
```

```
df['binned'] = pd.cut(df['numerical_column'], bins=5)
```

```
# Equal-frequency binning
```

```
df['binned'] = pd.qcut(df['numerical_column'], q=5)
```

- Use Case:** Converting continuous to categorical, handling non-linear relationships

4. Advanced Feature Engineering

A. Feature Creation

- Polynomial Features:** x^2 , x^3 interactions
- Interaction Terms:** Multiply two features
- Date/Time Features:** Day of week, month, season, is_weekend
- Text Features:** Word counts, sentiment scores, topic modeling
- Domain-Specific Features:** Business metrics, ratios, aggregates

B. Feature Selection

- Correlation Analysis:** Remove highly correlated features

- Variance Threshold:** Remove low-variance features
- Recursive Feature Elimination:** Iteratively remove weakest features
- Feature Importance:** Using tree-based models

5. Best Practices & Considerations

When to Use Which Technique:

- Tree-based models:** Less sensitive to scaling, can handle mixed types
- Distance-based models:** Always require scaling (KNN, SVM, K-means)
- Linear models:** Benefit from scaling and normalization
- Neural networks:** Require normalized inputs

Data Leakage Prevention:

- Fit scalers/encoders on training data only
- Transform test data using fitted objects
- Use pipelines to avoid leakage

Validation Strategy:

- Always validate feature engineering impact using cross-validation
 - Monitor for overfitting with target encoding
 - Test different encoding strategies
-

Correlations & Relationships

1. Understanding Correlation

Definition: Correlation measures the strength and direction of the linear relationship between two variables.

Key Concepts:

- Strength:** How strongly variables are related (-1 to 1)
- Direction:** Positive or negative relationship
- Linear vs Non-linear:** Correlation measures linear relationships only

2. Types of Correlation Measures

A. Pearson Correlation Coefficient

- **Measures:** Linear relationship between two continuous variables
- **Range:** -1 to +1
- **Formula:** $\text{Covariance}(X,Y) / (\sigma_X * \sigma_Y)$
- **Interpretation:**
 - +1: Perfect positive linear relationship
 - -1: Perfect negative linear relationship
 - 0: No linear relationship
- **Assumptions:** Linear relationship, normally distributed, no outliers

B. Spearman's Rank Correlation

- **Measures:** Monotonic relationship (linear or non-linear)
- **Uses:** Rank order of values
- **Range:** -1 to +1
- **Use Case:** Ordinal data, non-normal distributions, non-linear relationships

C. Kendall's Tau

- **Measures:** Ordinal association between two measured quantities
- **More robust** to outliers than Spearman
- **Use Case:** Small sample sizes, many tied ranks

D. Point-Biserial Correlation

- **Measures:** Relationship between continuous and binary variables
- **Essentially:** Pearson correlation where one variable is dichotomous

E. Phi Coefficient

- **Measures:** Association between two binary variables
- **Essentially:** Pearson correlation for 2x2 contingency tables

3. Correlation Analysis Techniques

A. Correlation Matrix

python

Calculate correlation matrix

```
correlation_matrix = df.corr(method='pearson')
```

Visualize with heatmap

```
plt.figure(figsize=(12, 8))
```

```
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,  
            square=True, linewidths=0.5)
```

```
plt.title('Correlation Matrix Heatmap')
```

```
plt.show()
```

B. Pairwise Correlation Analysis

python

Find highly correlated features

```
def find_high_correlations(df, threshold=0.8):
```

```
    corr_matrix = df.corr().abs()
```

```
    upper_triangle = np.triu(np.ones_like(corr_matrix, dtype=bool), k=1)
```

```
    high_corr = corr_matrix.where(upper_triangle) > threshold
```

```
    return corr_matrix.where(high_corr).stack().sort_values(ascending=False)
```

```
high_corr_pairs = find_high_correlations(df)
```

```
print("Highly correlated feature pairs:")
```

```
print(high_corr_pairs)
```

C. Correlation with Target Variable

python

Analyze correlations with target variable

```
target_correlations = df.corr()[['target_column']].sort_values('target_column', ascending=False)
```

```
print("Feature correlations with target:")
```

```
print(target_correlations)
```

4. Advanced Relationship Analysis

A. Partial Correlation

- Measures relationship between two variables while controlling for others
- **Use Case:** Remove confounding variable effects

B. Cross-Correlation

- For time series data
- Measures correlation at different time lags

C. Autocorrelation

- Correlation of a variable with itself at different time points
- Use Case:** Time series analysis, seasonality detection

5. Non-Linear Relationship Detection

A. Scatter Plots with Regression Lines

```
python  
  
# Scatter plot with regression line  
sns.jointplot(x='feature1', y='feature2', data=df, kind='reg')  
plt.show()
```

B. Residual Analysis

- Check if relationship is truly linear
- Patterns in residuals indicate non-linearity

C. Mutual Information

```
python  
  
from sklearn.feature_selection import mutual_info_regression  
  
# Measures both linear and non-linear dependencies  
mi_scores = mutual_info_regression(X, y)
```

6. Categorical-Numerical Relationships

A. ANOVA (Analysis of Variance)

- Tests if means of numerical variable differ across categorical groups
- Use Case:** Compare numerical variable across multiple categories

B. Chi-Square Test

- Tests association between two categorical variables
- Use Case:** Identify significant relationships between categories

C. Cramer's V

- Measures association between two categorical variables

- Range:** 0 (no association) to 1 (perfect association)

7. Practical Correlation Analysis Framework

Step-by-Step Approach:

- 1.**Compute Correlation Matrix:** All numerical variables
- 2.**Identify Strong Correlations:** Both positive and negative
- 3.**Visualize Relationships:** Heatmaps, scatter plots
- 4.**Check for Multicollinearity:** High correlations between features
- 5.**Analyze Target Relationships:** Features most related to target
- 6.**Investigate Non-linear Patterns:** Beyond correlation coefficients
- 7.**Validate with Domain Knowledge:** Do correlations make sense?

Red Flags to Watch For:

- Spurious Correlations:** Coincidental relationships
- Confounding Variables:** Hidden factors influencing both variables
- Non-linear Relationships:** Missed by correlation coefficients
- Outlier-driven Correlations:** Driven by few extreme points

8. Correlation Pitfalls & Limitations

Common Misinterpretations:

- Correlation $\neq$ Causation:** The most critical reminder
- Non-linear Relationships:** Correlation only measures linear relationships
- Outlier Sensitivity:** A few outliers can dramatically affect correlation
- Range Restriction:** Limited variable range can suppress correlation
- Ecological Fallacy:** Group-level correlations don't apply to individuals

When Correlation Can Be Misleading:

- Presence of outliers** that dominate the relationship
- Non-linear relationships** that appear weak linearly
- Subgroups** with different relationships within the data
- Time-based relationships** that change over different periods